

HTTP Client

Small HTTP requests, directly from DuckDB.



Install & load

```
INSTALL http_client FROM community;  
LOAD http_client;
```

01 At a glance

Fetch HTTP resources or submit JSON and form bodies from a SQL expression. Responses arrive as a JSON envelope containing status, reason and body, ready to inspect or save in a local table.

HTTP request → JSON envelope → SQL projection

02 Requests & response fields

`http_get(url)`

Fetch a response; a three-argument overload accepts maps.

`http_head(url)`

Request response metadata without downloading the body.

`http_post(url, headers, json)`

Send a JSON body; headers are `MAP(VARCHAR, VARCHAR)`.

`http_post_form(url, headers, params)`

Send form fields supplied as a `VARCHAR-to-VARCHAR` map.

`response->>'status'`

Read the HTTP status; cast to `INTEGER` for comparisons.

`response->>'body'`

Extract the body string; cast to `JSON` when appropriate.

03 Keep in mind

Materialize responses

Requests are synchronous and buffer the response. Save responses to a table before repeated analysis. Deduplicate input keys for enrichment: a function evaluation can trigger a network call, and SQL evaluation order is not a request scheduler.

04 Fetch once, inspect locally

REQUIRES SETUP

```
CREATE TEMP TABLE response AS  
SELECT http_get('https://example.com/') AS r;  
SELECT (r->>'status')::INTEGER AS status,  
       r->>'reason' AS reason,  
       r->>'body' AS body  
FROM response;
```

Requires network access. The example.com response is HTML, so keep body as text.

05 POST a JSON document

REQUIRES SETUP

```
SELECT http_post(  
  'https://api.example.com/events',  
  MAP {'content-type': 'application/json'},  
  {'event': 'ready', 'rows': 42}::JSON  
);
```

Replace the URL with your POST endpoint and add required authentication headers.

06 Decode a JSON response body

```
WITH r AS (  
  SELECT '{"status":200,  
         "body":{"count":42}}'::JSON AS v  
)  
SELECT ((v->>'body')::JSON)->>'count' AS n  
FROM r;
```

Offline envelope fixture; returns the text value 42.

07 In practice

Inspect status before parsing

A body can be HTML, plain text or JSON. Check status and expected content before casting it. Endpoint authentication, retries and rate limits belong in the surrounding workflow; do not assume an HTTP error is a successful API result.

[Full docs & examples ↗](#) [Source ↗](#)